

Lab Sheet 03

DG 958202 (Computer Graphics Programming)

ผศ. ดร. ชีรวัฒน์ กำเนิดศิริ

ชื่อ-นามสกุล _____ รหัส _____ Section _____

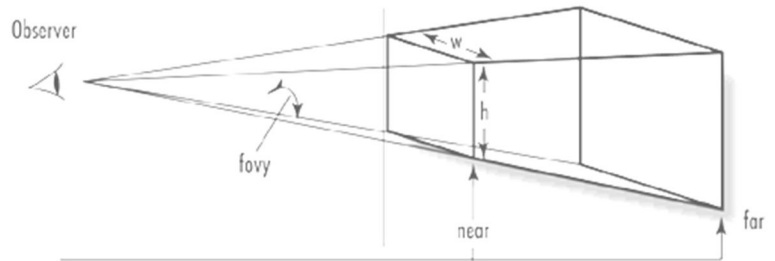
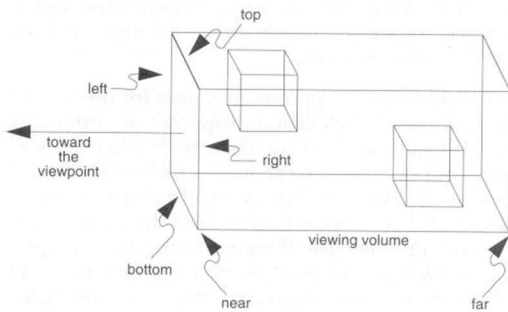
Three-Dimensional Viewing

จุดประสงค์เชิงพฤติกรรม

1. สามารถอธิบายความแตกต่างระหว่าง Orthographic Projections กับ Perspective Projections ได้
2. สามารถเขียนโปรแกรมโดยใช้ แสดงภาพบน Perspective Projections ได้

ให้นักศึกษาทำตามคำสั่งดังต่อไปนี้

1. จงอธิบายความแตกต่างของ Orthographic Projections กับ Perspective Projections และเขียนคำสั่งพร้อมยกตัวอย่างการใช้งานของแต่ละ Projection



2. เขียนโปรแกรมสำหรับแสดงผลภาพด้วย Perspective Projections (ตั้งชื่อไฟล์เป็นรหัสนักศึกษาLab3_1.cpp)

```
#include <windows.h>
#include <GL/glut.h>
void init(void);
void ChangeSize(void);
void RenderScene(void);

void init(void)
{
    glClearColor(0.0, 0.0, 0.0, 0.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-150.0, 150.0, -150.0, 150.0, -150.0, 150.0);
}

void ChangeSize(GLsizei w, GLsizei h)
{
    GLfloat fAspect;
    if(h == 0)
        h = 1;
    glViewport(0, 0, w, h);
    fAspect = (GLfloat)w/(GLfloat)h;
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(50.0f, fAspect, 1.0, 500.0);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

void RenderScene(void)
{
    glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    gluLookAt(1,1,1,0,0,0,0,1,1);

    GLfloat mat_specular[] = {1.0,1.0,1.0,1.0};
    GLfloat mat_diffuse[] = {1, 0.0, 0.0,1.0};
    GLfloat mat_ambient[] = {1.0,1.0,1.0,1.0};
    GLfloat mat_shininess[] = {10.0 };
    GLfloat light_position[] = {-1.0,2.0,0.0,0.0};
    GLfloat light_spec[] = { 1.0,1.0,1.0,0.0};
    GLfloat light_amb[] = { 1, 0.5,0.0,0.0};
    glClearColor(0.0,0.0,0.0,0.0);
    glShadeModel(GL_SMOOTH);
    glMaterialfv(GL_FRONT, GL_SPECULAR, mat_specular);
    glMaterialfv(GL_FRONT, GL_DIFFUSE, mat_diffuse);
    glMaterialfv(GL_FRONT, GL_AMBIENT, mat_ambient);
    glMaterialfv(GL_FRONT, GL_SHININESS, mat_shininess);
    glLightfv(GL_LIGHT1, GL_SPECULAR, light_spec);
    glLightfv(GL_LIGHT1, GL_AMBIENT, light_amb);
    glLightfv(GL_LIGHT1, GL_POSITION, light_position);
    glEnable(GL_LIGHTING);
    glEnable(GL_LIGHT1);
    glEnable(GL_DEPTH_TEST);
}
```

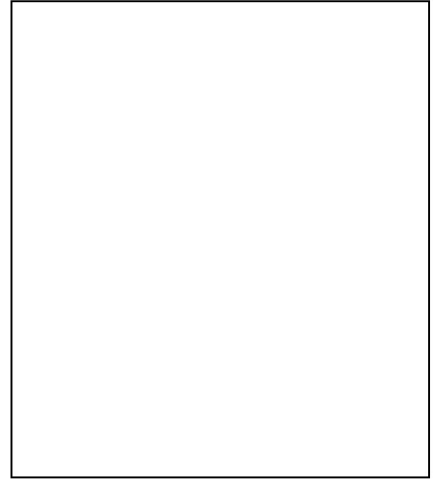
```

glPushMatrix();
glTranslatef(0.1f, 0.1f, 0.0f);
glutSolidCube(0.2f);
glPopMatrix();
glPushMatrix();
glTranslatef(0.2f, 0.5f, 0.2f);
glutSolidSphere(0.2, 100, 100);
glPopMatrix();

glPushMatrix();
glTranslatef(-0.2f, -0.5f, -0.8f);
glutSolidSphere(0.2, 100, 100);
glPopMatrix();

glFlush ();
glutSwapBuffers();
}

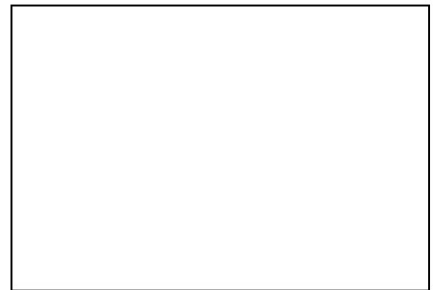
```



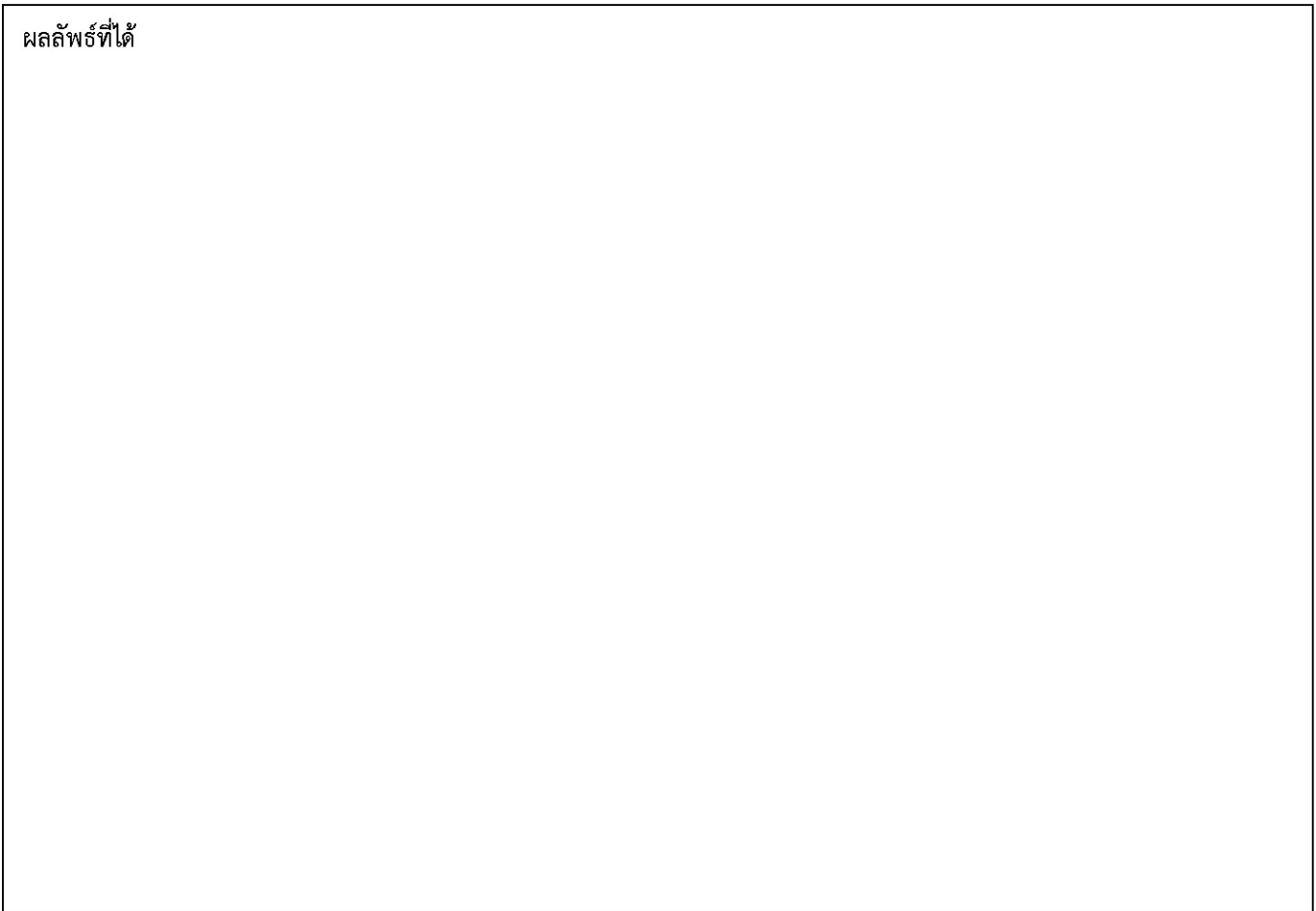
```

void main(int argc, char **argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE|GLUT_RGB|GLUT_DEPTH);
    glutInitWindowPosition(10,10);
    glutInitWindowSize(800,800);
    glutCreateWindow("Perspective Projections");
    init();
    glutDisplayFunc(RenderScene);
    glutReshapeFunc(ChangeSize);
    glutMainLoop();
}

```



ผลลัพธ์ที่ได้



3. เขียนโปรแกรมสำหรับแสดงผลภาพด้วย Perspective Projections (ตั้งชื่อไฟล์เป็นรหัสนักศึกษาLab3_2.cpp)

```
#include <windows.h>
#include <GL/glut.h>
#include <math.h>

#define PI 3.14159265358979324
using namespace std;

static int p = 6;
static int q = 4;
static float *vertices = NULL;
static float Xangle = 150.0, Yangle = 60.0, Zangle = 0.0;

float f(int i, int j)
{
    return ( cos( (-1 + 2*(float)i/p) * PI ) );
}

float g(int i, int j)
{
    return ( sin( (-1 + 2*(float)i/p) * PI ) );
}

float h(int i, int j)
{
    return ( -1 + 2*(float)j/q );
}

void fillVertexArray(void)
{
    int i, j, k;

    k = 0;
    for (j = 0; j <= q; j++)
        for (i = 0; i <= p; i++)
        {
            vertices[k++] = f(i,j);
            vertices[k++] = g(i,j);
            vertices[k++] = h(i,j);
        }
}

void setup(void)
{
    glEnableClientState(GL_VERTEX_ARRAY);
    glClearColor(1.0, 1.0, 1.0, 0.0);
}

void drawScene(void)
{
    int i, j;
    vertices = new float[3*(p+1)*(q+1)];

    glVertexPointer(3, GL_FLOAT, 0, vertices);
    glClear(GL_COLOR_BUFFER_BIT);

    glLoadIdentity();
    gluLookAt (0.0, 0.0, 4.0, 0.0, 0.0, 0.0, 1.0, 0.0);

    glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
    glColor3f(0.0, 0.0, 1.0);
```

```

glRotatef(Zangle, 0.0, 0.0, 1.0);
glRotatef(Yangle, 0.0, 1.0, 0.0);
glRotatef(Xangle, 1.0, 0.0, 0.0);

fillVertexArray();

for(j = 0; j < q; j++)
{
    glBegin(GL_TRIANGLE_STRIP);
    for(i = 0; i <= p; i++)
    {
        glVertexElement( (j+1)*(p+1) + i );
        glVertexElement( j*(p+1) + i );
    }
    glEnd();
}

glutSwapBuffers();
}

void resize(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(60.0, (float)w/(float)h, 1.0, 50.0);
    glMatrixMode(GL_MODELVIEW);
}

void keyInput(unsigned char key, int x, int y)
{
    switch(key)
    {
        case 27:
            exit(0);
            break;
        case 'x':
            Xangle += 5.0;
            if (Xangle > 360.0) Xangle -= 360.0;
            glutPostRedisplay();
            break;
        case 'X':
            Xangle -= 5.0;
            if (Xangle < 0.0) Xangle += 360.0;
            glutPostRedisplay();
            break;
        case 'y':
            Yangle += 5.0;
            if (Yangle > 360.0) Yangle -= 360.0;
            glutPostRedisplay();
            break;
        case 'Y':
            Yangle -= 5.0;
            if (Yangle < 0.0) Yangle += 360.0;
            glutPostRedisplay();
            break;
        case 'z':
            Zangle += 5.0;
            if (Zangle > 360.0) Zangle -= 360.0;
            glutPostRedisplay();
            break;
        case 'Z':
            Zangle -= 5.0;

```

```

        if (Zangle < 0.0) Zangle += 360.0;
        glutPostRedisplay();
        break;
    default:
        break;
    }
}

void specialKeyInput(int key, int x, int y)
{
    if (key == GLUT_KEY_LEFT) if (p > 3) p -= 1;
    if (key == GLUT_KEY_RIGHT) p += 1;
    if (key == GLUT_KEY_DOWN) if (q > 3) q -= 1;
    if (key == GLUT_KEY_UP) q += 1;

    glutPostRedisplay();
}

int main(int argc, char **argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGBA);
    glutInitWindowSize(500, 500);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("cylinder.cpp");
    glutDisplayFunc(drawScene);
    glutReshapeFunc(resize);
    glutKeyboardFunc(keyInput);
    glutSpecialFunc(specialKeyInput);
    setup();
    glutMainLoop();
}

```

ผลลัพธ์ที่ได้

4. เขียนโปรแกรมสำหรับแสดงผลภาพด้วย Camera view program (ตั้งชื่อไฟล์เป็นรหัสนักศึกษาLab3_3.cpp)

```
#include <windows.h>
#include <gl/glut.h>
#include <math.h>

float angle = 0.0, ratio;
float x = 0.0f, y = 0.0f, z = 20.0f;
float lx = 0.0f, ly = 0.0f, lz = -1.0f;

void drawTorus(void)
{
    glColor3f(0.0, 0.0, 1.0);
    glutWireTorus(0.5, 3, 25, 50);
}

void RenderScene(void)
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    drawTorus();
    glutSwapBuffers();
}

void init(void)
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
}

void reshape(int w, int h)
{
    if (h == 0)
        h = 1;

    ratio = 1.0f * w / h;
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glViewport(0, 0, w, h);
    gluPerspective(45, ratio, 1, 1000);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    gluLookAt(x, y, z, x + lx, y + ly, z + lz, 0.0f, 1.0f, 0.0f);
}

void orientMe(float ang) {
    lx = sin(ang);
    lz = -cos(ang);
    glLoadIdentity();
    gluLookAt(x, y, z, x + lx, y + ly, z + lz, 0.0f, 1.0f, 0.0f);
}
```

```

void moveMeFlat(int i) {
    x = x + i * (lx) * 1;
    z = z + i * (lz) * 1;
    glLoadIdentity();
    gluLookAt(x, y, z, x + lx, y + ly, z + lz, 0.0f, 1.0f, 0.0f);
}

void processNormalKeys(unsigned char key, int x, int y) {

    if (key == 27)
        exit(0);
}

void processSpecialKeys(int key, int x, int y) {
    switch (key) {
        case GLUT_KEY_LEFT: angle -= 0.2f; orientMe(angle); break;
        case GLUT_KEY_RIGHT: angle += 0.2f; orientMe(angle); break;
        case GLUT_KEY_UP: moveMeFlat(1); break;
        case GLUT_KEY_DOWN: moveMeFlat(-2); break;
    }
}

void main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGBA | GLUT_DEPTH);
    glutInitWindowPosition(100, 100);
    glutInitWindowSize(640, 480);
    glutCreateWindow("The camera viewing program");
    init();
    glutDisplayFunc(RenderScene);
    glutIdleFunc(RenderScene);
    glutReshapeFunc(reshape);
    glutKeyboardFunc(processNormalKeys);
    glutSpecialFunc(processSpecialKeys);
    glEnable(GL_DEPTH_TEST);
    glutMainLoop();
}

```

ผลลัพธ์ที่ได้

5. ให้นักศึกษาเขียนโปรแกรมสำหรับแสดงผลภาพ ชื่อตัวเองเป็นตัวอักษรภาษาอังกฤษ 3 มิติ สามารถเลือกสีในการแสดงผลได้ตามอิสระ (ตั้งชื่อไฟล์เป็นรหัสนักศึกษาLab3_4_a.cpp)

a. ให้นักศึกษาเติมคำสั่งของของตั้งค่าการ

รูปแบบของคำสั่ง

gluOrtho_____

ให้กำหนดค่า และแสดงผล (ให้กำหนดค่าเองโดยไม่ซ้ำกัน)

gluOrtho _____

ผลลัพธ์ที่ได้

b. ให้นักศึกษาเติมคำสั่งของของตั้งค่าการ

รูปแบบของคำสั่ง

gluPerspective_____

ให้กำหนดค่า และแสดงผล (ให้กำหนดค่าเองโดยไม่ซ้ำกัน)

gluPerspective _____

ผลลัพธ์ที่ได้

c. ให้นักศึกษาเติมคำสั่งของของตั้งค่าการ

รูปแบบของคำสั่ง

gluLookAt_____

กำหนดค่า และแสดงผล (ให้กำหนดค่าเองโดยไม่ซ้ำกัน)

gluLookAt _____

ผลลัพธ์ที่ได้

ลายเซ็นผู้ตรวจ _____ วันที่ _____